Informe: Algoritmos de Caminos Mínimos en Java

Taller 6 – Ciencias de la Computación 2

16 de octubre de 2026

1. Introducción

Este informe explica los tres algoritmos de búsqueda de caminos mínimos que incluye el programa: **Dijkstra**, **Bellman-Ford** y **Floyd-Warshall**. Los tres trabajan sobre el mismo grafo (clase `Grafo`), que puede ser dirigido o no dirigido y tiene un peso en cada relación.

El **camino mínimo** entre dos vértices es el camino cuya suma de pesos es la menor posible. Cada algoritmo se implementó en su propia clase y se ejecuta desde el menú principal.

2. Ideas básicas

Antes de ver cada algoritmo conviene conocer tres ideas que se repiten en los tres:

- **Distancia:** para cada vértice se guarda la mejor distancia conocida desde el origen. Al empezar es infinito, salvo el origen, que vale 0.
- **Relajación:** consiste en revisar si pasar por una relación mejora una distancia. Si $dist(u) + peso(u, v) < dist(v)$, entonces se actualiza $dist(v)$.
- **Predecesor:** junto con cada distancia se guarda el vértice anterior en el camino, y con eso se reconstruye la ruta completa al final.

Ejemplo usado en el informe

Se usará un grafo dirigido de cuatro vértices con estas relaciones:

Origen	Destino	Peso
A	B	4
A	C	1
C	B	2
B	D	5
C	D	8

3. Algoritmo de Dijkstra

3.1. Idea

Calcula los caminos mínimos desde **un solo vértice de origen** hacia todos los demás. En cada paso elige el vértice no procesado con la menor distancia y relaja sus relaciones.

3.2. Funcionamiento en el código

1. Construye una lista de adyacencia a partir de las relaciones.
2. Comprueba que no haya pesos negativos. Si los hay, se detiene y avisa.
3. Inicializa todas las distancias en infinito, salvo el origen en 0.
4. Usa una cola de prioridad (`PriorityQueue`) para sacar siempre el vértice de menor distancia.
5. Relaja las relaciones del vértice sacado y agrega a la cola los que mejoran.
6. Si una entrada de la cola ya quedó desactualizada (su distancia es peor que la guardada), la ignora.

3.3. Ejemplo desde A

Paso	**Elegido**	**A**	**B**	**C**	**D**
Inicio	–	0	∞	∞	∞
1	A	0	4	1	∞
2	C	0	3	1	9
3	B	0	3	1	8
4	D	0	3	1	8

En el paso 2, pasar por C mejora B de 4 a 3. En el paso 3, pasar por B mejora D de 9 a 8. El camino mínimo de A a D es A $\rightarrow$ C $\rightarrow$ B $\rightarrow$ D, con distancia 8.

3.4. Limitación

Dijkstra supone que sumar una relación nunca acorta un camino, por eso **no funciona con pesos negativos**.

4. Algoritmo de Bellman-Ford

4.1. Idea

También calcula los caminos mínimos desde un solo origen, pero es más lento y más general: **admite pesos negativos** y **detecta ciclos negativos**.

4.2. Funcionamiento en el código

1. Convierte las relaciones en una lista de aristas. En un grafo no dirigido agrega cada relación en los dos sentidos.
2. Inicializa las distancias igual que Dijkstra.
3. Recorre **todas** las aristas y las relaja, y repite esto como máximo $V - 1$ veces (V es el número de vértices). Si en una vuelta nada cambia, termina antes.
4. Hace una vuelta extra. Si todavía se puede mejorar alguna distancia, existe un **ciclo negativo** alcanzable desde el origen.

4.3. Ejemplo con peso negativo

Sea un grafo dirigido con A$\rightarrow$B de peso 4, A$\rightarrow$C de peso 5 y B$\rightarrow$C de peso -3. Desde A, la distancia a C es $4 + (-3) = 1$, que es menor que ir directo (5). Bellman-Ford lo encuentra sin problema.

4.4. Ciclo negativo

Un ciclo negativo es un recorrido que vuelve al mismo vértice y cuya suma es negativa. Por ejemplo, A→B (2), B→C (−1), C→A (−3) suma −2. Cada vuelta baja la distancia, así que no existe un mínimo. En ese caso el programa muestra un aviso y no imprime distancias.

5. Algoritmo de Floyd-Warshall

5.1. Idea

Calcula los caminos mínimos entre **todos los pares de vértices** de una sola vez. Para cada vértice k se pregunta: ¿ir de i a j pasando por k es mejor que el camino actual?

5.2. Funcionamiento en el código

1. Crea una matriz de distancias: 0 en la diagonal, el peso de la relación donde existe y infinito en el resto.
2. Crea una matriz `siguiente`, que guarda el próximo vértice en el camino de i a j y sirve para reconstruir las rutas.
3. Aplica tres ciclos anidados sobre k, i y j:

$$dist[i][j] = \text{mín}(dist[i][j],\ dist[i][k] + dist[k][j])$$

4. Revisa la diagonal: si algún $dist[i][i]$ es negativo, hay un ciclo negativo.
5. Muestra la matriz de distancias y el camino de cada par.

5.3. Ejemplo

Con el grafo del ejemplo, la matriz final de distancias mínimas es (el guion indica que no hay camino):

	A	B	C	D
A	0	3	1	8
B	–	0	–	5
C	–	2	0	7
D	–	–	–	0

Por ejemplo, de C a D la distancia es 7 y no 8, porque conviene ir C → B → D (2 + 5) en lugar de la relación directa. La primera fila coincide con lo que calculó Dijkstra desde A.

6. Comparación

	Dijkstra	**Bellman-Ford**	**Floyd-Warshall**
Resultado	Un origen	Un origen	Todos los pares
Pesos negativos	No	Sí	Sí
Detecta ciclos negativos	No	Sí	Sí
Complejidad	$O((V+E)\log V)$	$O(V \cdot E)$	$O(V^3)$
Estructura principal	Cola de prioridad	Lista de aristas	Matrices

¿Cuándo usar cada uno?

- **Dijkstra:** cuando los pesos son no negativos y solo se necesita un origen. Es el más rápido.

- **Bellman-Ford:** cuando puede haber pesos negativos o se quiere detectar ciclos negativos desde un origen.
- **Floyd-Warshall:** cuando se necesitan las distancias entre todos los pares y el grafo es pequeño o mediano.

7. Consideraciones sobre el programa

- En un **grafo no dirigido** no se aceptan pesos negativos, porque una sola relación negativa ya forma un ciclo negativo (se puede ir y volver por ella indefinidamente). Por eso los ciclos negativos solo aparecen en grafos dirigidos.
- Cuando no hay camino entre dos vértices, los tres algoritmos lo indican con el mensaje "No existe camino".
- Dijkstra y Bellman-Ford piden un vértice de origen. Floyd-Warshall no lo necesita porque calcula todos los pares.

8. Conclusión

Los tres algoritmos resuelven el mismo problema con distintas estrategias. Dijkstra es el más eficiente, pero solo sirve con pesos no negativos. Bellman-Ford sacrifica velocidad para aceptar pesos negativos y detectar ciclos negativos. Floyd-Warshall entrega todas las distancias a la vez usando matrices. Elegir uno depende de si hay pesos negativos y de si se necesita un solo origen o todos los pares.